

# Resolution of the FinalCorruptZip Forensic Challenge

**Author:** David Liu, Bennett Plumb, Devkumar Banerjee

## Challenge Overview

The FinalCorruptZip challenge involves repairing a corrupted ZIP archive and a modified PNG file to recover a hidden flag (SVBGR{m4g1C\_B1t3s\_yUmmmmm}). The solution requires correcting file signatures (magic bytes) at both the ZIP and PNG levels. Below is a detailed breakdown of the resolution process.

## Tools and Prerequisites

Hex Editors:

HxD (Windows), xxd/hexedit (Linux), or Bless (macOS/Linux) for binary file analysis.  
Critical for direct byte-level modifications.

Archive Utilities:

7-Zip, WinZip, or command-line tools like unzip (Linux/macOS) for extraction.

Image Validation Tools:

Built-in OS image viewers or tools like IrfanView to confirm PNG integrity.

## Step 1: Repairing the ZIP File Header

### Background

ZIP files are identified by their magic bytes 50 4B 03 04 (ASCII: PK..). In this challenge, the first byte (50, representing "P") was altered to 41 ("A"), rendering the file unreadable by standard extractors.

### Procedure

Initial Analysis:

Open corrupted.zip in a hex editor.

Observe the first four bytes: 41 4B 03 04 (incorrect) instead of 50 4B 03 04 (correct).

Byte Correction:

Navigate to offset 0x00 (first byte).

Replace 41 with 50 to restore the ZIP signature.

Save the modified file as repaired.zip.

Validation:

Attempt extraction using `unzip repaired.zip` (Linux/macOS) or 7-Zip (Windows).

Successful extraction yields a file named image.png. If extraction fails, re-verify the hex values or check for additional corruption.

## Step 2: Repairing the PNG Signature

Background

PNG files begin with an 8-byte signature: 89 50 4E 47 0D 0A 1A 0A. The first four bytes in this challenge were altered to 65 4C 4F 4C (ASCII: eLOL), which must be corrected to 65 50 4E 47 (ePNG) to restore readability.

Procedure

Hex Analysis of image.png:

Open the extracted image.png in a hex editor.

Search for the sequence 65 4C 4F 4C at offset 0x00.

Signature Correction:

Replace the first four bytes as follows:

Original (corrupted): 65 4C 4F 4C

Corrected: 65 50 4E 47

Save the modified file as repaired\_image.png.

Validation:

Open repaired\_image.png in an image viewer. If the flag is not visible:

Confirm the full 8-byte PNG header: 89 50 4E 47 0D 0A 1A 0A.

Use file repaired\_image.png (Linux/macOS) to verify the file type.

## Flag Retrieval

Upon successful repair of both files, open repaired\_image.png to view the flag:

SVBGR{m4g1C\_B1t3s\_yUmmmmm}

## Technical Insights and Lessons Learned

Magic Bytes Significance:

File parsers rely on magic bytes to identify formats. Even minor alterations (e.g., changing P to A) disrupt parsing.

Hex Editing Best Practices:

Always create backups before modifying files.

Use checksums (e.g., sha256sum) to detect unintended changes.

Systematic Validation:

Test fixes incrementally. For example, repair the ZIP first, confirm extraction, then address the PNG.

#### Recommendations for Further Skill Development

Practice file forensics on platforms like CyberChef or CTF sites (e.g., CTFtime).

Study common file signatures (e.g., PDF: 25 50 44 46, JPEG: FF D8 FF E0).

#### **Conclusion**

This challenge underscores the importance of understanding low-level file structures and the role of magic bytes in digital forensics. By methodically correcting corrupted headers, analysts can recover data even from severely damaged files.